- unholdTickets, the reverse of holdTicket, happens when the user clicks on an already held seat. It uses filter to create a new list of myTickets without the unheld seats, then makes that new list part of the state.

- clearHolds, called when the user clicks the new holds button and sets the myTickets list to an empty list

In all of these cases, the useReducer hook causes our components to be re-rendered when the state changes, so all of these changes are immediately visible in the display. Specifically, for our purposes, the calculation of the subtotal, which is based on the length of the myTickets array, changes when the state changes.

We need to define two types, (1) the VenueState type that has all the state information, and (2) the VenueAction type that the reducer uses to determine which action has been called. We've put both these types in their own file, called venue_types.ts.

Here's the state data declaration:

chapter_11/01/app/javascript/contexts/venue_types.ts

```
export interface TicketData {
  id: number
  number: number
  row: number
  status: string
}

export interface VenueState {
  concertId: number
  myTickets: TicketData[]
  otherTickets: TicketData[]
  rowCount: number
  seatsPerRow: number
  ticketsToBuyCount: number
}
```

For the state, we have the same row count, seats per row, concert ID, and tickets-to-buy count that we've been using. We also add the list of tickets coming in